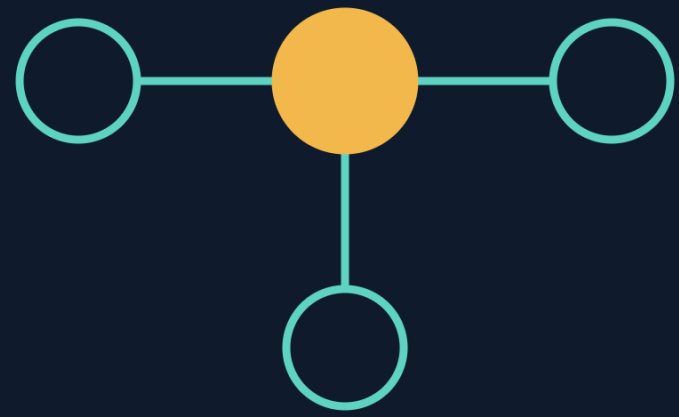
Assets/Scripts/AStar/NavGraph.cs

NavGraph

Un grafo de navegación guardado en arrays planos: cómo funciona, de punta a punta.



Qué necesita el A* de un mapa

`Vector3[] positions`

¿Dónde está?

Cada nodo tiene una posición en el mundo. Sirve para dibujar el grafo y para estimar cuánto falta hasta la meta.

`List<int>[] neighbors`

¿Con quién conecta?

Cada nodo tiene una lista de vecinos: los nodos a los que se puede ir directo desde él.

`List<float>[] neighborsCost`

¿Cuánto cuesta ir?

Cada conexión tiene un costo, por ejemplo la distancia. El A* busca el camino de menor costo total.

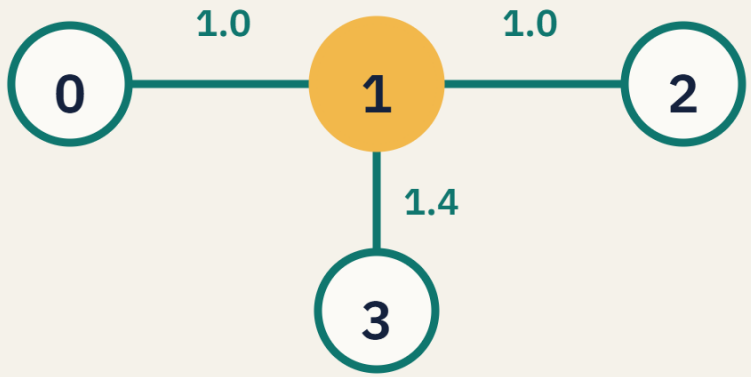
Un nodo no es un GameObject: es solo un número entero, su **índice** (0, 1, 2…).

Cuatro arrays lo guardan todo

Campo	Tipo	Tamaño	Qué guarda
_positions	Vector3[]	N	La posición de cada nodo
_neighborStart	int[]	N + 1	Dónde empieza, en _neighbors, el tramo de vecinos de cada nodo
_neighbors	int[]	E	Los vecinos de todos los nodos, uno tras otro
_neighborCosts	float[]	E	El costo de cada una de esas conexiones

N = cantidad de nodos. E = cantidad total de conexiones, contando cada sentido por separado.

Un ejemplo: grafo de 4 nodos



Nodo	Vecinos	Costos
0	1	1.0
1	0, 2, 3	1.0, 1.0, 1.4
2	1	1.0
3	1	1.4

Cada conexión se guarda dos veces, una desde cada extremo. Por eso hay 3 conexiones dibujadas pero E = 6.

Cómo se arma _neighborStart

Vecinos por nodo

nodo 0	nodo 1	nodo 2	nodo 3
1	3	1	1

`_neighborStart`

nodo 0	nodo 1	nodo 2	nodo 3	fin
0	1	4	5	6

`_neighbors`

pos 0	pos 1	pos 2	pos 3	pos 4	pos 5
1	0	2	3	1	1

```
start[0] = 0 · start[i + 1] = start[i] + vecinos del nodo i
```

El tramo del nodo 1 va de start[1] = 1 hasta start[2] − 1 = 3 (en ámbar).

Tamaño y copia: una pasada por nodo

```
int totalEdges = _neighborStart[positions.Length];
_neighbors = new int[totalEdges];
_neighborCosts = new float[totalEdges];

for (int i = 0; i < positions.Length; i++)
{
    neighbors[i].CopyTo(_neighbors, _neighborStart[i]);
    neighborsCost[i].CopyTo(_neighborCosts, _neighborStart[i]);
}
```

- El último valor de _neighborStart es el total de conexiones: define el tamaño de los dos arrays planos.
- El tramo del nodo i empieza en _neighborStart[i].
- CopyTo pega la lista de ese nodo justo ahí.
- Ejemplo: el nodo 1 escribe en las posiciones 1, 2 y 3.

Leer los vecinos de un nodo

`GetNeighborCount(node)` Cuántos vecinos tiene el nodo.	`_neighborStart[node + 1] - _neighborStart[node]` Nodo 1: 4 – 1 = 3 vecinos.
`GetNeighbor(node, k)` El k-ésimo vecino del nodo.	`_neighbors[_neighborStart[node] + k]` Nodo 1, k = 2: posición 1 + 2 = 3, que guarda el nodo 3.
`GetNeighborCost(node, k)` El costo de ir a ese vecino.	`_neighborCosts[_neighborStart[node] + k]` Nodo 1, k = 2: la misma posición 3, costo 1.4.

k es la posición dentro del tramo del nodo (de 0 a count – 1). **No es el índice de un nodo.**

Cómo se va a usar desde el A* (todavía no está escrito)

El A* solo pregunta, no conoce los arrays

```
int count = graph.GetNeighborCount(current);

for (int k = 0; k < count; k++)
{
    int next = graph.GetNeighbor(current, k);
    float cost = graph.GetNeighborCost(current, k);
    // nuevoG = g[current] + cost
}
```

- El for recorre k de 0 a count – 1.
- Con cada k obtiene el vecino y lo que cuesta llegar a él.
- Si cambia cómo se guardan los vecinos, el A* no se entera.

FindNearestNode: del mundo al grafo

Un enemigo está en un punto del mundo, no en un índice. Para pedir un camino hay que traducir esa posición al nodo más cercano.

1

Recorre

Pasa por los N nodos, uno por uno.

2

Compara

Calcula (posición − worldPos).sqrMagnitude: la distancia al cuadrado, sin raíz cuadrada.

3

Devuelve

El índice del nodo con menor distancia, o −1 si el grafo no tiene nodos.

Cuesta una pasada completa por todos los nodos en cada consulta. Alcanza para empezar; con grafos grandes se reemplaza por una búsqueda más rápida.

Por qué arrays planos y no un GameObject por nodo

Un GameObject por nodo

- Cada nodo es un objeto de la escena.
- La búsqueda pasa por Transform y componentes.
- Miles de objetos dispersos en memoria.

Arrays planos

- Datos contiguos en memoria.
- El A* trabaja con enteros y con índices.
- No se reserva memoria durante la búsqueda.

Contrapartida: el grafo se arma una vez y no se modifica. Si el nivel cambia, hay que reconstruirlo.

Cosas a tener en cuenta

La validación solo avisa

Debug.LogError no frena el constructor. Si las listas no coinciden, el código sigue y puede fallar con una excepción o dejar costos mal asignados.

Sin chequeo de rangos

Un k fuera de 0 … count – 1 en GetNeighbor o GetNeighborCost lee el tramo de otro nodo, o se sale del array.

Listas nunca null

Un nodo sin vecinos necesita una lista vacía, con su lista de costos vacía también.

Se llena una sola vez

Los arrays se arman en el constructor y no cambian después. Marcar los campos como readonly haría que el compilador lo asegure.

Lo que sigue

- **GridGraphBuilder**: llena positions, neighbors y neighborsCost para una grilla plana.
- **MinHeap**: la cola de prioridad del A*.
- **AStarPathfinder**: recorre el grafo con GetNeighborCount, GetNeighbor y GetNeighborCost.

Antes de seguir, probá NavGraph con el grafo de 4 nodos de esta presentación.

`GetNeighborCount(1) = 3 · GetNeighbor(1, 2) = 3 · suma de los counts = 6`